Lab Assignment 2

Cache Analysis

David Åkerlind

Contents

1	**Introduction**	**3**
2	**Code conversion**	**3**
3	**Cache analysis**	**6**
4	**Problems**	**10**
5	**Summary**	**10**

1 Introduction

This assignment consisted of two parts. The first part involved converting a C program to MIPS assembly code inside the MARS environment by implementing several subroutines and a main program. The program prints an array, sorts it using merge sort, and prints the sorted array afterwards.

The second part involved analysing the cache memory performance when running just the merge sort portion of the program. Three series of experiments were performed

- A direct-mapped cache with a fixed size of 128 bytes, varying block sizes (1, 2, 4, 8, 16 and 32 words), for array sizes 10 and 20.

- A direct-mapped cache with a fixed block size of 16 bytes, varying cache sizes (16, 32, 64, 128, 256, 512 and 1024 bytes), for array sizes 10 and 20.

- An N-way set-associative cache (LRU replacement policy) with cache size 128 bytes and block size 16 bytes, varying N (1, 2, 4, 8), for array sizes 10 and 20. The effect of using random instead of LRU replacement was also examined.

In this report, cache hit rate refers to the percentage of memory accesses that were found in the cache rather than requiring access to main memory (RAM).

2 Code conversion

The code conversion from C to MIPS assembly required careful planning of program structure, stack management, and register usage. The approach taken was to implement each C function as a separate MIPS subroutine, following the calling convention used by GCC.

The main program file contains the global variables (size and vec[]) in the .data section, along with the format strings used for printing. Each subroutine allocates shadow space on the stack at the start and saves any callee-saved registers ($s0-$s6, $ra) that it uses

Stack Management

Each subroutine begins by allocating stack space using subu $sp, $sp, N, where N accounts for shadow space and saved registers. For example, the merge subroutine allocates 52 bytes for 8 saved registers plus $ra:

```
merge:
    subu $sp, $sp, 52
    sw $ra, 48($sp)
    sw $s0, 44($sp)
    sw $s1, 40($sp)
    sw $s2, 36($sp)
    ...
```

The merge subroutine also dynamically allocates the local array b[] on the stack, calculating the required bytes as size * 4 and subtracting from $sp. This allocation happens after saving the registers, so the saved register offsets remain stable. The space is freed before returning by adding the same amount back to $sp.

Register Usage

The $s registers ($s0-$s6) were used to hold values that needed to persist across subroutine calls, such as the base addresses of arrays, loop counters (i, j, k), and the size and half variables. The $t registers were used for temporary calculations such as computing array offsets. Arguments were passed in $a0-$a3 and return values in $v0-$v1.

In retrospect, some use of $s registers was unnecessary. The base address of a[] and the size value are already preserved in the $a registers for the duration of a call, so storing them again in $s registers served no purpose. Removing these redundant saves would reduce stack usage, which in turn could have slightly improved the cache results by reducing the program's overall memory footprint. This is one area that would be improved in a future revision.

Array elements were accessed using offset addressing rather than incrementing base pointers, which avoided the need to restore base addresses after loops. For example, to access b[i]:

```
sll  $t0, $s4, 2          # i * 4
addu $t0, $s3, $t0        # address of b[i]
lw   $t1, 0($t0)          # $t1 = b[i]
```

Recursive mergeSort

The mergeSort subroutine is recursive, which required saving $ra on the stack before each jal call and restoring it before returning. Without this, the return address would be overwritten on recursive calls, causing incorrect program flow, as for all jal calls in the program.

3 Cache analysis

Series 1 — Varying Block Size (Direct-Mapped, 128 bytes)

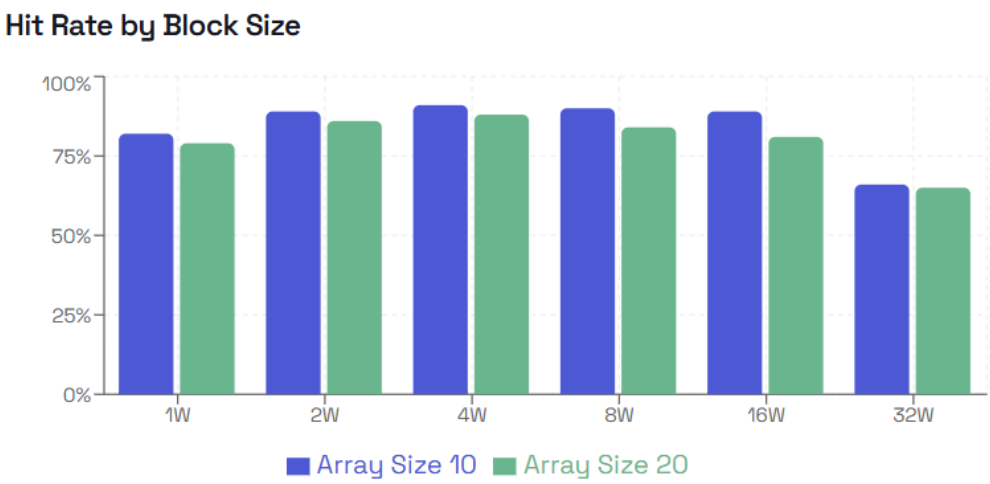
Settings: direct-mapped cache, cache size = 128 bytes, block sizes = 1, 2, 4, 8, 16, 32 words.

Block Size Hit Rates

Cache size = 128 bytes · Enter hit rate % for each configuration

Block Size	# Blocks	Size = 10	Size = 20
1 words	32	82	79
2 words	16	89	86
4 words	8	91	88
8 words	4	90	84
16 words	2	89	81
32 words	1	66	65

Table 1: Cache hit rates for varying block sizes (direct-mapped, 128 bytes)



Graph 1: Cache hit rates for varying block sizes (direct-mapped, 128 bytes)

The results show that hit rate peaks at a block size of 4 words (91% for size=10, 88% for size=20), then decreases for larger blocks. Increasing block size improves spatial locality since more neighbouring array elements are loaded on each cache miss, which benefits the sequential access patterns in merge sort. However, with a fixed cache size of 128 bytes, larger blocks mean fewer total cache blocks. At 32 words per block only 1 block fits in the entire cache, causing a sharp drop to 66% and 65%. The larger array (size=20) consistently shows lower hit rates since it occupies more memory and increases the chance of evicting useful data.

Series 2 — Varying Cache Size (Direct-Mapped, 16-byte blocks)

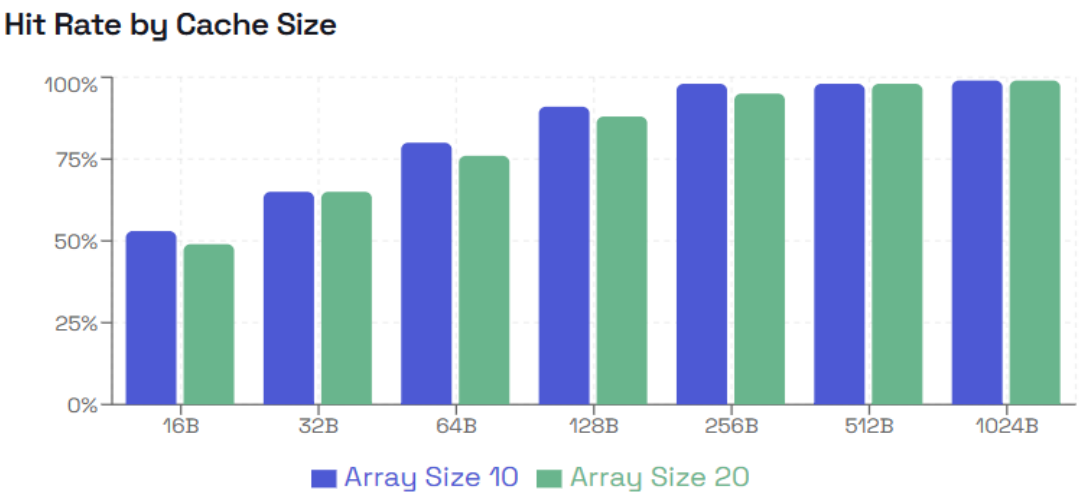
Settings: direct-mapped cache, block size = 16 bytes (4 words), cache sizes = 16 to 1024 bytes.

Cache Size Hit Rates

Enter hit rate % for each cache size (bytes)

Cache Size	Size = 10	Size = 20
16 B	53	49
32 B	65	65
64 B	80	76
128 B	91	88
256 B	98	95
512 B	98	98
1024 B	99	99

Table 2: Cache hit rates for varying cache sizes (direct-mapped, 4-word blocks)



Graph 2: Cache hit rates for varying cache sizes (direct-mapped, 4-word blocks)

Hit rate increases steadily as cache size grows, rising from 53% at 16 bytes to 99% at 1024 bytes for size=10. This is expected since a larger cache holds more data simultaneously, reducing capacity and conflict misses. The improvement is steep at smaller sizes (16B to 128B) and flattens at larger sizes (512B and above), suggesting that at 512 bytes the cache is large enough to hold most of the working data for merge sort. The size=20 array shows lower hit rates at smaller cache sizes but catches up at 512B and 1024B where both arrays fit comfortably in cache.

Series 3 — Varying Associativity (N-way Set-Associative)

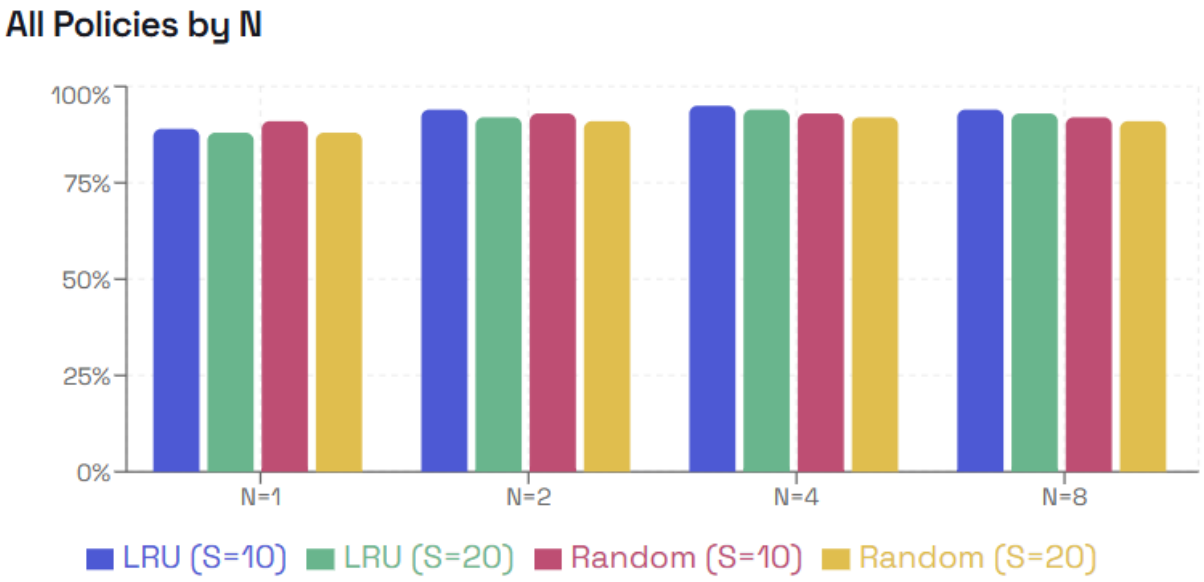
Settings: N-way set-associative, LRU replacement, cache size = 128 bytes, block size = 16 bytes (4 words).

Set Associativity Hit Rates

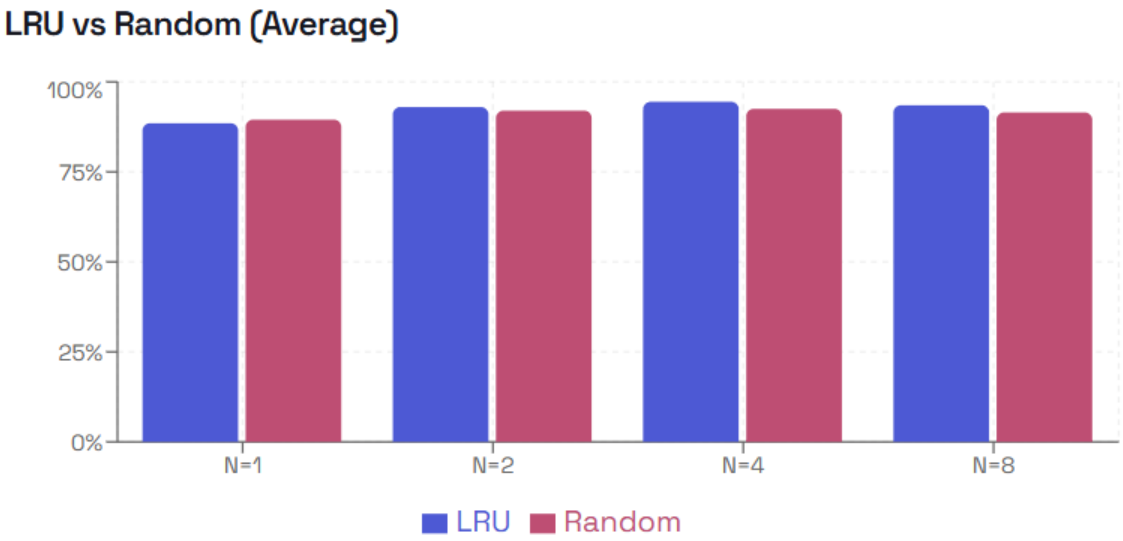
Enter hit rate % for LRU and Random replacement

N	LRU		Random	
	S=10	S=20	S=10	S=20
1	89	88	91	88
2	94	92	93	91
4	95	94	93	92
8	94	93	92	91

Table 3: Cache hit rates for varying associativity N with LRU and Random replacement



Graph 3: Cache hit rates for varying associativity N with LRU and Random replacement



Graph 4: Average (size 10 and 20) cache hit rates for varying associativity N with LRU and Random replacement

Increasing associativity from N=1 to N=4 improves hit rate under LRU, rising from 89% to 95% for size=10. This is because higher associativity reduces conflict misses — more blocks can coexist in the same set without evicting each other. At N=8 the hit rate drops slightly to 94%, likely due to the overhead of managing more ways with a small working set. Comparing LRU and random replacement, the results are surprisingly close. At N=1 random actually scores slightly higher (91% vs 89% for size=10), but for N=2 and above

LRU is equal or better. This suggests that for small N, LRU does not have a strong advantage, but as N grows the smarter eviction policy of LRU better preserves recently accessed data. The larger array (size=20) consistently shows slightly lower hit rates across all settings.

4 Problems

- Stack corruption: The most significant problem was a stack offset mismatch caused by allocating the dynamic b[] array on the stack after saving registers. This caused incorrect values to be loaded when restoring $ra, leading to an invalid program counter error (0x10010000). The fix was to save all registers first with stable offsets, then allocate b[] separately to the stack.

- Label errors: Some jump instructions referenced incorrect or mistyped labels, causing infinite loops or jumps to wrong locations. Careful naming conventions (m_while1, m_while2, m_while3) resolved this. Return labels presented a similar challenge, as their purpose can be non-obvious and naming them clearly requires care.

- Missing register saves: Initially $s2-$s6 were not saved on the stack, which caused their values to be corrupted across nested subroutine calls. Adding sw/lw for all used $s registers fixed this. But this comes with the price of needing more memory space allocated on the stack. Attempts were made to minimize stack usage but this could have been done much better.

- More stack corruption: Insufficient shadow space allocation caused additional issues across several subroutines. As shadow space was an unfamiliar concept initially, the required amounts were underestimated. This was resolved by systematically calculating the exact stack space needed for each subroutine rather than estimating. Referring to the original C code proved useful here, as the function signatures made it straightforward to determine the correct allocation sizes.

5 Summary

This lab assignment provided hands-on experience with MIPS assembly programming and cache memory analysis. The code conversion part reinforced understanding of calling conventions, stack management, and register usage in low-level programming.

The cache analysis demonstrated how block size, cache size, and associativity each affect cache performance. In general, larger block sizes improve spatial locality, larger caches reduce capacity misses, and higher associativity reduces conflict misses.

Appendix

The program was developed in MARS using four separate source files: the main program, and the subroutines `merge_sort`, merge, and `print_array`. In the main file, these are imported under the `.text` section using `.include` directives, alongside the provided print subroutine. For the purposes of this report, all subroutines have been consolidated into a single listing and placed after the `.data` section but before the main program, following the convention used in C. The code presented here has been tested and produces the expected output.

```
          .data
#vector: .word 4,5,2,-2,1,6,7,9,-5,10,99,32,98,42,12,41,29,22,77,62
vector: .word 4,5,2,-2,1,6,7,9,-5,10
size: .word 10
format: .asciiz "%d "        # Format for print.s
new_line: .asciiz "\n"


          .text
          .globl main
          .include "print.s"
          #.include "print_array.s"
          #.include "merge_sort.s"
          #.include "merge.s"


print_array:
          li $t1, 36
          subu $sp, $sp, $t1           # make some space for storing and printing

          sw $ra, 20($sp)                        # store the seach-subrutine (this one) return
adress

# Store all of the $s regitsters because they are not to be changed after subrutines
# and we can use them without fear of teh print subroutine to write over them
          sw $s2, 24($sp)
          sw $s3, 28($sp)
          sw $s4, 32($sp)

# Load the arguments in to safe regristers
          li $s2, 0                                      # $s2 = start index
          move $s3, $a0           # $s3 = base address of vector = $s3
    move $s4, $a1           # $s4 = $a1 = vector size = $s4

for:
          bge $s2, $s4 end_loop   # if gone trough all elements in vector now end

    la $a0, format          # Load format string address
    lw $a1, 0($s3)          # Load the number to print = The next
    jal print               # Call the print subroutine

    addi $s2, $s2, 1                   # index++
          addi $s3, $s3, 4             # move to next number in array (4 bytes)

          j for

end_loop:

          la $a0, new_line
          jal print

          lw $ra, 20($sp)          # restore return address
    lw $s2, 24($sp)          # restore saved registers
    lw $s3, 28($sp)
    lw $s4, 32($sp)
```

```
    addu $sp, $sp, 36        # restore stack pointer

          jr $ra

merge_sort:
          subu $sp, $sp, 32             # make space for some variables

          sw $ra, 28($sp)                         # store the seach-subrutine (this one) return
adress
          sw $s0, 24($sp)                         # save base adress for array to be able to use
it in different calls
          sw $s1, 20($sp)

          move $s0, $a0            # $s0 = base address of vector = $s0
    move $s1, $a1            # $t1 = $a1 = vector size = $t1

          ble $s1, 1 ms_return          # Compare the SIZE with 1 :: IF size > 1: continue,
ELSE: end

          divu $t2, $s1, 2              # int half = size / 2 = $t2 = HALF

# PREPARE mergeSort(a, half);
          move $a0, $s0            # $t0 = base address of vector = $t0
    move $a1, $t2            # $t2 = $a1 = size / 2

          jal merge_sort                          # mergeSort(a, half = $t2 );

# PREPARE  mergeSort(a + half, size - half);
    divu $t2, $s1, 2         # recalculate half
    mul  $t3, $t2, 4         # byte offset = half * 4
    addu $a0, $s0, $t3       # a + half (in bytes)
    subu $a1, $s1, $t2       # size - half

          jal merge_sort                          # mergeSort(a + half, size - half);

# PREPARE For MERGE
          move $a0, $s0            # $s0 = base address of vector = $a0
    move $a1, $s1            # $s1 = $a1 = size
          jal merge                               #  merge(a, size);

          j ms_return


ms_return:
    lw $ra, 28($sp)
    lw $s0, 24($sp)
    lw $s1, 20($sp)   # restore $s1
    addu $sp, $sp, 32
    jr $ra


merge:
          subu $sp, $sp, 52
    sw $ra, 48($sp)
    sw $s0, 44($sp)
    sw $s1, 40($sp)
    sw $s2, 36($sp)
    sw $s3, 32($sp)
    sw $s4, 28($sp)
    sw $s5, 24($sp)
    sw $s6, 20($sp)

          move $s0, $a0                           # base a[] = $s0
          move $s1, $a1                           # size = $s1

# half = $s2
          div $s2, $s1, 2              # half = $s2

# b[] = $s3
          sll $t0, $s1, 2         # $t0 = size * 4($t3 is teh size we need for b[])
          subu $sp, $sp, $t0      # allocate $t0 space b[] on stack
          move $s3, $sp           # $s3 = base address of b[]
```

```
# i = $t4
          li $t4, 0

m_for:
    bge  $t4, $s1, m_end_for
    sll  $t0, $t4, 2        # i * 4
    addu $t1, $s0, $t0      # address of a[i]
    lw   $t2, 0($t1)        # t2 = a[i]
    addu $t3, $s3, $t0      # address of b[i]
    sw   $t2, 0($t3)        # b[i] = a[i]
    addi $t4, $t4, 1        # i++
    j    m_for

# $s0 = base address of a[]
# $s1 = size
# $s2 = half
# $s3 = base address of b[]
# $s4 = i
# $s5 = j
# $s6 = k


# this restores some values to continue the merge
m_end_for:
    li   $s4, 0             # i = 0
    move $s5, $s2           # j = half
    li   $s6, 0             # k = 0

m_while1: #  while (i < half && j < size)
          bge $s4, $s2, m_while2 # i < half
          bge $s5, $s1, m_while2 # j < size

# Load b[i] = $s3[$s4] in to $t1
    sll  $t0, $s4, 2         # i * 4
    addu $t0, $s3, $t0       # address of b[i]
    lw   $t1, 0($t0)         # $t1 = b[i]

# Load b[j] = $s3[$s5] in to $t3
          sll  $t2, $s5, 2         # j * 4
    addu $t2, $s3, $t2       # address of b[j]
    lw   $t3, 0($t2)         # $t3 = b[j]

          bgt  $t1, $t3, m_else     # if b[i] > b[j], go to else


m_if: # b[i] <= b[j]
          move $t4, $s4            # i
    addi $s4, $s4, 1         # i++
    j    m_end_if


m_else:  # b[i] > b[j]
    move $t4, $s5            # j
          addi $s5, $s5, 1         # j++

m_end_if:
          sll $t0, $t4, 2              # index * 4
          addu $t0, $s3, $t0
          lw, $t1, 0($t0)              # Load b[index]
          sll  $t2, $s6, 2        # k * 4
    addu $t2, $s0, $t2
    sw   $t1, 0($t2)        # a[k] = b[index]

    addi $s6, $s6, 1         # k++
          j          m_while1


# $s0 = base address of a[]
# $s1 = size
# $s2 = half
# $s3 = base address of b[]
```

```
# $s4 = i
# $s5 = j
# $s6 = k

m_while2: # Task a[k] = b[i]    IF: while (i < half)
    bge  $s4, $s2, m_while3

    sll  $t0, $s4, 2                  # calculate the index based i * 4 = $t0
    addu $t0, $s3, $t0                # save the start adress of b[i]
    lw   $t1, 0($t0)        # save value of int b[i] in $t1

    sll  $t2, $s6, 2                  # calculate the index k * 4 = $t2
    addu $t2, $s0, $t2                # save the start adress of a[k]

    sw   $t1, 0($t2)        # a[k] = b[i] / copy the numbers over
    addi $s4, $s4, 1        # i++
    addi $s6, $s6, 1        # k++
    j    m_while2


m_while3: # TASK: a[k] = b[j]   IF: while (j < size)
    bge  $s5, $s1, m_return

    sll  $t0, $s5, 2                  # calculate the index based j * 4 = $t0
    addu $t0, $s3, $t0                # save the start adress of b[j]
    lw   $t1, 0($t0)        # b[j] save the value of b[j]

    sll  $t2, $s6, 2                  # calculate the index based k * 4 = $t2
    addu $t2, $s0, $t2                # save the start adress of a[k]

    sw   $t1, 0($t2)        # a[k] = b[j]
    addi $s5, $s5, 1        # j++
    addi $s6, $s6, 1        # k++
    j    m_while3


m_return:
    sll  $t3, $s1, 2                  # calculate the size of b[] (to now how much we used9
    addu $sp, $sp, $t3      # free b[] space

    lw $s6, 20($sp)         # restore s0-s6 and $ra before returning
    lw $s5, 24($sp)
    lw $s4, 28($sp)
    lw $s3, 32($sp)
    lw $s2, 36($sp)
    lw $s1, 40($sp)
    lw $s0, 44($sp)
    lw $ra, 48($sp)
    addu $sp, $sp, 52                 # remove shadow space
    jr $ra


main:
          subu $sp, $sp, 16
# Call search function (passing 2 arguments)     jal print_array
          # Load vecor and size in to parameters
          la $a0, vector          # $a0 = base address of array
          lw $a1, size
          jal print_array

# Merge sort call (int a[], int size)
          la $a0, vector          # $a0 = base address of array
          lw $a1, size
          jal merge_sort

# Call search function (passing 2 arguments)     jal print_array
          # Print array call (int a[], int size)
          la $a0, vector          # $a0 = base address of array
          lw $a1, size
          jal print_array

# End program
```

```
        addu $sp, $sp, 16
        li $v0, 10
        syscall
```